

توثيق مشروع لوحة إدارة النقل

تحليل معماري وبرمجي مبسط للمبتدئين، مع شرح التقنيات، تدفق البيانات، نقاط القوة، القيود، وخطة العرض العملي.

جاهز للمراجعة

Lint و Build تم التحقق منهما

API خارجي

fetchApi عبر HTTP requests

Frontend SPA

React + TypeScript + Vite

فهرس المحتويات

Project Overview .2 02

Executive Summary .1 01

Project Structure .4 04

Technologies .3 03

Workflow and Data Flow .6 06

System Architecture .5 05

Frontend .8 08

Detailed Code Explanation .7 07

Database .10 10

Backend .9 09

Hardware and Communication .12 12

AI .11 11

Testing .14 14

Installation and Running .13 13

Limitations and Future Work .16 16

Security and Code Quality .15 15

Demo Plan .18 18

Competition Presentation Script .17 17

Glossary .20 20

Expected Questions and Answers .19 19

Final Cheat Sheet .21 21

تم توليد هذا الملف من التوثيق العربي بعد تحليل ملفات المشروع. الأقسام غير الموجودة في الكود الحالي موضحة بصراحة داخل الوثيقة.

توثيق مشروع TransPay Dashboard

تاريخ التحليل: 29-06-2026

نطاق التحليل: ملفات مشروع Dashboard/ باستثناء node_modules و dist والملفات المولدة.
ملاحظة مهمة: هذا المستودع يحتوي على Frontend فقط. لا يوجد Backend أو قاعدة بيانات داخل الكود المحلي، بل توجد طبقة خدمات تتصل بـ API خارجي موثق في endpoints/API-Admin.md و endpoints/API.md.

1. Executive Summary

المشروع هو لوحة تحكم إدارية باسم **TransPay** لإدارة منصة نقل مرتبطة برحلات المطار. الواجهة تسمح للمدير بتسجيل الدخول، متابعة مؤشرات عامة، إدارة الشركات، إدارة السائقين، عرض العملاء، عرض حجوزات نقل المطار، وحسابات التسويات المالية.

الفكرة الرئيسية هي تحويل عمليات الإدارة اليدوية لشركات النقل والسائقين والحجوزات إلى لوحة مركزية يمكن من خلالها البحث، التصفية، الحذف، إضافة شركة، تسجيل سائق، ومتابعة مؤشرات مالية وتشغيلية.

النظام مبني كتطبيق **Single Page Application (SPA)** باستخدام React و TypeScript و Vite. يعتمد على API خارجي عبر HTTP، ويحفظ رمز الدخول token داخل sessionStorage. لا يوجد في المشروع الحالي ذكاء اصطناعي، ولا هاردوير، ولا قاعدة بيانات محلية، ولا Backend محلي.

أهم ملاحظة يجب الانتباه لها أمام اللجنة: بعض الشاشات تعتمد على API حقيقي مثل الشركات والسائقين والعملاء، بينما بعض الأجزاء تعتمد على بيانات وهمية أو endpoints غير مؤكدة، مثل حجوزات المطار في src/features/dashboard/LiveMap.tsx وخريطة السائقين في src/services/adminService.ts.

2. Project Overview

اسم المشروع

الاسم الظاهر في الواجهة هو **TransPay**، ويظهر في:

- `src/pages/LoginPage.tsx`
- `src/layouts/Sidebar.tsx`
- تقرير PDF داخل `src/pages/Dashboard.tsx`
- اسم الحزمة في `package.json` هو `dashboard`.

المشكلة التي يحاول المشروع حلها

إدارة منصة نقل تحتوي على شركات وسائقين وعملاء وحجوزات قد تصبح معقدة إذا تمت يدويًا أو عبر ملفات منفصلة. المشروع يحاول توفير لوحة واحدة للمدير لمتابعة العمليات اليومية، مثل:

- معرفة عدد الشركات النشطة.
- معرفة إجمالي الحركة المالية.
- إدارة الشركات.
- إدارة السائقين حسب الشركة.
- عرض العملاء وحذفهم.
- عرض حجوزات النقل بين البيت والمطار.
- تقدير التسويات بين المنصة والشركات/السائقين.

الفئة المستهدفة

الفئة الأساسية هي مدير النظام أو فريق العمليات في منصة نقل. هناك أيضًا صفحة عامة لتسجيل السائقين عبر `/driver-register`، لكن الكود الحالي لا يربطها فعليًا بتدفق OTP رغم وجود دوال OTP في

`src/services/adminService.ts`.

سيناريو عملي

1. يدخل المدير إلى `/login`.
2. يكتب رقم/هوية الهاتف وكلمة المرور.

3. التطبيق يرسل الطلب إلى `/Auth/admin/login`.
4. إذا نجح الدخول، يتم حفظ `token` في `sessionStorage`.
5. ينتقل المدير إلى لوحة التحكم.
6. يستطيع فتح صفحة الشركات وإضافة شركة أو حذف شركة.
7. يستطيع فتح صفحة السائقين، اختيار شركة، ثم تسجيل سائق جديد مع صور الهوية والمركبة.
8. يستطيع متابعة الحجوزات أو التسويات المالية.

القيمة مقارنة بالطريقة التقليدية

بدل متابعة الشركات والسائقين والحجوزات يدويًا، تجمع الواجهة أهم العمليات في مكان واحد مع بحث، صفحات، مؤشرات، جداول، وطلبات API منظمة. هذا يقلل الوقت المطلوب لإدارة البيانات، ويجعل عرض الحالة العامة للمنصة أسرع وأوضح.

Technologies .3

التقنية أو الأداة	مكان استخدامها	وظيفتها	سبب استخدامها في المشروع
TypeScript	معظم ملفات <code>src/**/*.ts</code> و <code>src/**/*.tsx</code>	إضافة أنواع ثابتة للكود	تقليل أخطاء التعامل مع بيانات API و Props
React	<code>src/App.tsx</code> , <code>src/pages</code> , <code>src/features</code>	بناء واجهة المستخدم كمكونات	مناسب لبناء SPA تفاعلي
Vite	<code>vite.config.ts</code> , <code>package.json</code>	تشغيل وبناء المشروع	سريع في التطوير ويدعم React بسهولة
React Router DOM	<code>src/App.tsx</code> , <code>src/pages/DriverRegisterPage.tsx</code>	التنقل بين الصفحات	إدارة مسارات مثل <code>/login</code> , <code>/drivers</code>
Zustand	<code>src/store/*</code>	إدارة حالة بسيطة	مناسب للتنبيهات وبعض بيانات Mock بدون Redux معقد
Tailwind CSS	<code>src/index.css</code> , <code>tailwind.config.js</code>	تنسيق الواجهة	بناء تصميم سريع ومتسق عبر Utility Classes
Base UI	<code>src/components/ui/Button.tsx</code> , <code>dialog.tsx</code>	primitives للأزرار والحوارات	يوفر أساس Accessible لبعض عناصر UI
class-variance-authority	<code>Button.tsx</code> , <code>Badge.tsx</code>	Variants للمكونات	توحيد أشكال الأزرار والشارات
clsx + tailwind-merge	<code>src/lib/utils.ts</code>	دمج CSS classes	منع تعارض classes في Tailwind
lucide-react	معظم الصفحات والمكونات	أيقونات	تحسين وضوح الواجهة بصريًا
Recharts	<code>src/features/dashboard/RevenueChart.tsx</code>	رسم مخطط الإيرادات	عرض البيانات المالية بصورة مرئية
Leaflet + React Leaflet	<code>src/features/dashboard/LiveMap.tsx</code>	خريطة تفاعلية	عرض مواقع السائقين، حاليًا ببيانات ثابتة
jsPDF	<code>src/pages/Dashboard.tsx</code>	إنشاء ملف PDF	تصدير تقرير مبسط من لوحة التحكم
react-datepicker	<code>src/pages/Dashboard.tsx</code>	اختيار نطاق تاريخ	فلتر/عرض الفترة الزمنية في الهيدر

التقنية أو الأداة	مكان استخدامها	وظيفتها	سبب استخدامها في المشروع
Fetch API	<code>src/lib/apiClient.ts</code>	إرسال HTTP requests	الاتصال بالـ Backend الخارجي
sessionStorage	<code>src/lib/apiClient.ts</code> , <code>src/App.tsx</code> , <code>LoginPage.tsx</code>	حفظ token مؤقتًا	إبقاء جلسة المدير أثناء فتح المتصفح
ESLint	<code>eslint.config.js</code>	فحص جودة الكود	اكتشاف أخطاء نمطية قبل البناء
npm	<code>package.json</code> , <code>package-lock.json</code>	إدارة الحزم	تنصيب وتشغيل Dependencies

شرح مبسط للتقنيات

React هو مكتبة لبناء الواجهات على شكل Components. في هذا المشروع كل صفحة مثل `CompaniesPage` ومكون مثل `CompanyList` هو جزء مستقل يمكن قراءته وتطويره وحده. البديل الممكن هو Vue أو Angular. الفرق المختصر أن React يعطي مرونة كبيرة، بينما Angular يأتي بإطار أكبر وأكثر صرامة.

TypeScript هو JavaScript مع Types. مثلًا `DriverModel` في `src/types/admin.ts` يوضح شكل بيانات السائق. لو لم يستخدم المشروع TypeScript، ستكون أخطاء مثل استخدام حقل غير موجود أصعب في الاكتشاف أثناء التطوير.

Vite هو أداة تشغيل وبناء. يوفر Server للتطوير عبر `npm run dev` ويبنى ملفات الإنتاج عبر `npm run build`. بديله Webpack أو Next.js. Vite أخف للمشروع الحالي لأنه Frontend فقط.

Zustand مكتبة State Management بسيطة. استخدمت هنا للتنبيهات `useToastStore`، وليبيانات Mock قديمة في `useCompanyStore` و `useTripStore`. بديله Redux، لكنه أكبر وأكثر تعقيدًا.

Tailwind CSS يسمح بكتابة التنسيق مباشرة في `className`. لو لم يستخدم، كان المشروع سيحتاج ملفات CSS كثيرة أو مكتبة UI جاهزة.

Fetch API هي واجهة المتصفح لإرسال طلبات HTTP. المشروع يلفها داخل `fetchApi` في `src/lib/apiClient.ts` حتى لا تتكرر إضافة headers ومعالجة الأخطاء في كل ملف.

Project Structure .4

```
Dashboard/
├── endpoints/
│   ├── API.md
│   └── API-Admin.md
├── public/
│   ├── favicon.svg
│   └── icons.svg
├── src/
│   ├── assets/
│   ├── components/ui/
│   ├── features/
│   │   ├── bookings/
│   │   ├── companies/
│   │   ├── customers/
│   │   ├── dashboard/
│   │   ├── drivers/
│   │   ├── settlements/
│   │   └── trips/
│   ├── layouts/
│   ├── lib/
│   ├── pages/
│   ├── services/
│   ├── store/
│   ├── types/
│   ├── App.tsx
│   ├── main.tsx
│   └── index.css
├── package.json
├── vite.config.ts
├── tailwind.config.js
├── tsconfig.json
├── eslint.config.js
└── test-render.tsx
```

وظيفة المجلدات الرئيسية

المسار	الوظيفة
src/main.tsx	نقطة تشغيل React داخل عنصر <code>root</code> في HTML
src/App.tsx	تعريف Routes وحماية الصفحات الخاصة
src/pages/	صفحات كاملة مربوطة بالمسارات
src/features/	منطق ومكونات الميزات الرئيسية مثل الشركات والسائقين

المسار	الوظيفة
src/components/ui/	مكونات UI مشتركة مثل Button و Badge و Modal
src/layouts/	الهيكل العام للوحة: TopBar و Sidebar
src/services/	دوال الاتصال بال API أو إرجاع بيانات مؤقتة
src/lib/	Utilities مثل API client و logger
src/store/	Zustand stores
src/types/	types و TypeScript interfaces
endpoints/	توثيق API خارجي وليس Backend محلي
public/	ملفات عامة مثل favicon

نقطة البداية

نقطة البداية الفعلية هي:

```
src/main.tsx -> src/App.tsx -> Routes -> Pages -> Features
```

ملف `index.html` يحتوي على عنصر `root` الذي يتم حقن React داخله.

System Architecture .5

المشروع يستخدم معمارية:

- **Client-Server Architecture:** Frontend يتصل بخادم خارجي.
- **Single Page Application:** التنقل يتم داخل React Router.
- **Component-Based Architecture:** الواجهة مقسمة إلى Components.
- **REST API Client:** الطلبات ترسل عبر HTTP endpoints.

لا توجد معمارية Microservices داخل هذا المستودع، ولا يوجد MVC كامل لأن Backend غير موجود هنا.

```
User / Admin
↓
React Frontend (Vite SPA)
↓
ProtectedRoute checks sessionStorage token
↓
Pages and Feature Components
↓
services/*
↓
fetchApi in src/lib/apiClient.ts
  ↓ HTTP + Authorization: Bearer token
External API: https://aqaariq.com/marketplace/api/v1
↓
JSON response
↓
React state updates
↓
Tables / Charts / Toasts / Forms
```

مصادقة المدير

```
LoginPage
  ↓ adminLogin(phoneNumber, password)
apiClient.fetchApi('/Auth/admin/login', 'POST', body)
↓
External API returns token
↓
sessionStorage.setItem('token', token)
↓
window.location.href = '/'
↓
ProtectedRoute يسمح بالدخول
```

6. Workflow and Data Flow

دورة تشغيل التطبيق

1. يتم تشغيل Vite عبر `npm run dev`.
2. يفتح المتصفح تطبيق React.
3. `src/main.tsx` ينشئ React Root.
4. `src/App.tsx` يحدد المسار الحالي.
5. إذا كان المسار محميًا، `ProtectedRoute` يفحص وجود `token`.
6. إذا لم يوجد `token`، يتم التحويل إلى `/login`.
7. بعد تسجيل الدخول، يتم حفظ `token`.
8. كل طلب API لاحق يضيف `Authorization: Bearer <token>`.
9. البيانات تعرض داخل جداول أو بطاقات أو مخططات.
10. في حالة الخطأ، تسجل التفاصيل في Console فقط في بيئة التطوير عبر `logError`.

تدفق بيانات تسجيل الدخول

المرحلة	الملف	ماذا يحدث
إدخال البيانات	<code>src/pages/LoginPage.tsx</code>	المستخدم يكتب <code>password</code> و <code>phoneNumber</code>
إرسال الطلب	<code>src/lib/apiClient.ts</code>	<code>adminLogin</code> يستدعي <code>fetchApi</code>
حفظ الجلسة	<code>LoginPage.tsx</code>	يحفظ <code>response.token</code> في <code>sessionStorage</code>
حماية الصفحات	<code>src/App.tsx</code>	<code>ProtectedRoute</code> يمنع الدخول إذا لا يوجد <code>token</code>
انتهاء الجلسة	<code>src/lib/apiClient.ts</code>	عند 401 يتم حذف <code>token</code> والتحويل إلى <code>/login</code>

تدفق بيانات الشركات

```
CompanyList
  ↓ pageNum/pageSize/term
getCompanies(query)
  ↓ buildPaginationParams
fetchApi('/Company?...')
  ↓
setCompanies(response.data)
  ↓
Table + PaginationControls
```

الأخطاء المحتملة:

- فشل الشبكة أو API.
- اختلاف endpoint بين الكود وتوثيق `endpoints/API-Admin.md`.
- عدم عرض رسالة خطأ للمستخدم عند فشل جلب الشركات، لأن الكود يسجل الخطأ فقط في Console.

تدفق بيانات تسجيل السائق

```
DriverRegisterPage
  ↓
load companies from /Company
  ↓
user fills personal + vehicle data
  ↓
ImageUploadField validates image type and max 5 MB
  ↓
registerDriver builds FormData
  ↓
POST /Driver
  ↓
navigate('/drivers?companyId=...')
```

شكل البيانات المرسله هنا ليس JSON، بل `FormData` لأن الطلب يحتوي صوراً.

تدفق بيانات الحجوزات

حالياً `BookingList` يستدعي:

```
getAirportTransferBookings()
```

لكن هذه الدالة في `src/services/adminService.ts` تعيد مصفوفة ثابتة `bookingFixtures` ولا تتصل بـ API. لذلك هذا القسم Preview وليس تكاملاً كاملاً مع Backend.

Detailed Code Explanation .7

src/main.tsx

وظيفته تشغيل React داخل الصفحة:

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```

نحتاجه لأنه يربط تطبيق React بملف HTML. بدون هذا الملف لن تظهر الواجهة.

src/App.tsx

هذا الملف هو مركز التنقل والحماية.

أهم الاستيرادات:

- `react-router-dom` من `BrowserRouter`, `Routes`, `Route`, `Navigate`
- `lazy`, `Suspense` من React لتأخير تحميل الصفحات.
- `MainLayout` لتغليف صفحات الإدارة.
- `LoginPage` كصفحة عامة.

أهم الأجزاء:

- `Spinner`: `PageLoader` يظهر أثناء تحميل `Lazy pages`.
- `ProtectedRoute`: يفحص `sessionStorage.getItem('token')`.
- `Lazy loading` للصفحات مثل `Dashboard` و `Companies` و `Drivers`.

لماذا نحتاجه؟ لأنه يحدد ماذا يرى المستخدم حسب المسار وحسب حالة تسجيل الدخول.

خطأ محتمل: الحماية تعتمد فقط على وجود `token` في `sessionStorage`، ولا تتحقق من صلاحية `token` إلا عندما يرد الخادم بـ 401.

وظيفته توحيد كل طلبات API.

الثوابت المهمة:

- rawApiBase : يقرأ VITE_API_BASE_URL من البيئة.
- API_BASE : إذا لم يوجد env يستخدم https://aqaariq.com/marketplace/api/v1 .

أهم دالة:

```
export async function fetchApi<T>(endpoint: string, method = 'GET', body?: unknown): Promise<T>
```

المدخلات:

- endpoint : مثل /Company .
- method : GET أو POST أو PUT أو DELETE .
- body : البيانات المرسله.

ما تفعله:

1. تقرأ token من sessionStorage .
2. تحدد headers .
3. إذا كان body من نوع FormData لا تضع Content-Type حتى يضيف المتصفح boundary تلقائيًا.
4. تضيف Authorization لكل الطلبات ما عدا /auth .
5. ترسل الطلب عبر fetch .
6. عند 401 تحذف token وتعيد المستخدم إلى /login .
7. تحاول قراءة JSON .
8. إذا فشل الطلب ترمي Error يحتوي status و data و url .

لماذا مهم؟ لأنه يمنع تكرار منطق المصادقة ومعالجة الأخطاء في كل خدمة.

هذا هو ملف الخدمات الأساسي للإدارة.

أهم مجموعات الدوال:

المجموعة	الدوال	الوظيفة
Customers	getCustomers , getCustomer , updateCustomer , deleteCustomer	إدارة العملاء
Drivers	getDriver , getDriversByCompany , createDriver , updateDriver , deleteDriver , registerDriver	إدارة السائقين وتسجيلهم
Bookings	getAirportTransferBookings	يعيد بيانات حجوزات ثابتة حاليًا
OTP	requestDriver0tp , verifyDriver0tp , requestCustomer0tp , verifyCustomer0tp	دوال موجودة لكنها غير مستخدمة في صفحة التسجيل الحالية
Companies	getCompanies , getCompany , createCompany , updateCompany , deleteCompany	إدارة الشركات

ملاحظة مهمة: ملفات API في endpoints/API-Admin.md تستخدم أمثلة مثل /drivers و /companies ، بينما الكود يستخدم /Driver ، /Company ، /Customer . يجب التأكد من المسارات الحقيقية في الخادم.

src/pages/LoginPage.tsx

وظيفته عرض نموذج تسجيل دخول المدير.

الحالة المستخدمة:

- phoneNumber : رقم/هوية الدخول.
- password : كلمة المرور.
- error : رسالة خطأ.
- isLoading : حالة انتظار.

الدالة الأساسية:

```
const handleLogin = async (e: React.FormEvent) => { ... }
```

تمنع إعادة تحميل الصفحة، تستدعي adminLogin ، تحفظ token، ثم تحول المستخدم إلى / .

خطأ محتمل: رسالة الخطأ عامة جدًا ولا تعرض سبب الفشل الحقيقي من الخادم.

src/pages/Dashboard.tsx

تعرض الصفحة الرئيسية للوحة التحكم.

الأجزاء:

- اختيار نطاق تاريخ عبر `react-datepicker`.
- زر تصدير PDF عبر `jsPDF`.
- لبطاقات المؤشرات. `OverviewCards`
- للمخطط. `RevenueChart`
- للخريطة، يتم تحميلها Lazy. `LiveMap`

دالة `handleExport` تنشئ PDF بمعلومات ثابتة جزئيًا مثل `Active Vendors` و `Total Trips`. هذه الأرقام ليست محسوبة من API داخل هذه الدالة.

`src/features/dashboard/OverviewCards.tsx`

يجلب الرحلات والشركات عبر:

- `getTrips()`
- `getCompanyDirectory()`

ثم يحسب:

- `totalGmv`: مجموع أسعار الرحلات.
- `totalCommission`: 15% من GMV.
- `activeCompaniesCount`: عدد الشركات النشطة.
- `totalTripsToday`: الرحلات التي يبدأ وقتها بتاريخ اليوم.

ملاحظة: `/trips` endpoints و `/companies` تأتي من `dashboardService.ts` وليست نفس مسارات Admin الجديدة `/Trip` أو `/Company`. لا يمكن الجزم أنها تعمل دون اختبار API فعلي.

`src/features/dashboard/RevenueChart.tsx`

يعرض مخطط `AreaChart` باستخدام `Recharts`. البيانات تأتي من `getDashboardChart()` على endpoint `/dashboard/chart`. إذا لم توجد بيانات، تظهر رسالة عربية بأن البيانات غير كافية.

`src/features/dashboard/LiveMap.tsx`

يعرض خريطة `Leaflet` مع مواقع سائقين ثابتة داخل `driverLocations`.

مهم أمام اللجنة: الخريطة ليست Real-time فعلية حاليًا، رغم النص الظاهر في الواجهة. لا يوجد `WebSocket` أو `GPS` أو API لجلب المواقع داخل هذا الملف.

src/features/companies/CompanyList.tsx

يعرض جدول الشركات مع:

- بحث عبر `term`.
- `PaginationControls` عبر `Pagination`.
- `CompanyFormModal` عبر إضافة/تعديل.
- `ConfirmDialog` عبر حذف.
- `isSoftDeleted` عبر تمييز الحذف الناعم.

الدالة `fetchCompaniesData` تجلب البيانات من API ثم تحدث `companies` و `totalCount`.

src/features/companies/CompanyFormModal.tsx

نموذج إضافة أو تعديل شركة.

الحقول المرسلَة فعليًا:

- `name`
- `status` عند التعديل.
- `reputationScore` إذا كان رقمًا.

يوجد في الواجهة حقول إضافية مثل البريد والعنوان والسجل التجاري، لكنها غير مربوطة بـ `state` ولا ترسل إلى API. يجب عدم تقديمها كلجنة كميزة مكتملة.

src/features/drivers/DriverList.tsx

يعرض السائقين حسب الشركة المختارة.

تدفقه:

1. يجلب الشركات من `/Company`.
2. يختار الشركة الأولى أو `companyId` من URL.
3. يجلب السائقين عبر `/Driver/ByCompany/{companyId}`.
4. يسمح بحذف السائق غير المحذوف.
5. يفتح `/driver-register?companyId=...` لتسجيل سائق جديد.

src/pages/DriverRegisterPage.tsx

صفحة عامة لتسجيل السائق.

الحقول المطلوبة:

- الاسم.
- رقم الهاتف.
- الشركة.
- ماركة السيارة.
- موديل السيارة.
- رقم اللوحة.
- صورة الهوية الأمامية.
- صورة الهوية الخلفية.
- صورة تسجيل المركبة.
- ثلاث صور للمركبة بالضبط.

الدالة `handleRegister` تتحقق من الحقول والصور، ثم تستدعي `registerDriver`.

ملاحظة: رغم وجود تعليق في `App.tsx` يقول `OTP -> verify -> register` ، الصفحة الحالية لا تطلب OTP ولا تتحقق منه.

`src/features/drivers/ImageUploadField.tsx`

مكون رفع صور.

أهم منطق:

- يقبل صور PNG/JPEG/WebP من `input`.
- يتحقق أن النوع يبدأ بـ `image/`.
- يتحقق أن الحجم لا يتجاوز 5MB.
- يستخدم `URL.createObjectURL` للمعاينة.
- يستخدم `URL.revokeObjectURL` عند التنظيف لتقليل استهلاك الذاكرة.

`src/features/customers/CustomerList.tsx`

يعرض العملاء مع بحث و `Pagination` وحذف. يستخدم:

- `getCustomers`
- `deleteCustomer`
- `isSoftDeleted`

لا توجد شاشة تعديل عميل مربوطة حالياً رغم وجود `updateCustomer` في الخدمة.

`src/features/bookings/BookingList.tsx`

يعرض حجوزات نقل المطار:

- فلاتر حسب الاتجاه: الكل، البيت إلى المطار، المطار إلى البيت.
- بحث في رقم الحجز واسم العميل ورقم الهاتف ورقم الرحلة والمواقع.
- عرض السعر بالدينار العراقي IQD.

المصدر حاليًا بيانات ثابتة في `bookingFixtures`.

`src/features/settlements/FinancialSettlement.tsx`

يحسب التسويات من الرحلات المكتملة:

- `cashCollected`: المبالغ النقدية التي يحتفظ بها السائق.
- `onlinePaymentsHeld`: المدفوعات الإلكترونية لدى المنصة.
- `platformCommission`: 15% من إجمالي الحركة.
- `netBalance`: الفرق بين المدفوعات الإلكترونية والعمولة.

الأزرار مثل Download Report موجودة بصريًا لكنها لا تنفذ تصديرًا فعليًا داخل هذا الملف.

`src/layouts/MainLayout.tsx`, `Sidebar.tsx`, `TopBar.tsx`

`MainLayout` يضع `Sidebar` ثابت و `TopBar` ومحتوى الصفحة و `ToastContainer`.

`Sidebar` يربط الصفحات:

- Dashboard
- Drivers
- Customers
- Bookings
- Companies

لا يوجد رابط `Settings` أو `Settlements` في `Sidebar` رغم وجود صفحات لها في الكود. كذلك `SettlementsPage` موجودة في `App route`، لكنها لا تظهر في `Sidebar`.

`TopBar` يحتوي `Search input` وجرس إشعارات، لكن البحث والإشعارات غير مربوطة بمنطق فعلي.

`src/store/useToastStore.ts`

`Zustand store` للتنبيهات. عند إضافة `Toast` يتم توليد `id` عشوائي، ثم يحذف تلقائيًا بعد 4 ثوان.

`src/store/useTripStore.ts` و `src/store/useCompanyStore.ts`

تحتوي بيانات Mock ودوال لتحديثها. الاستخدام الفعلي الحالي محدود:

- `CompanyDetailsDrawer` يستخدم `useCompanyStore`.
- `CompanyDetailsDrawer` يستخدم `useTripStore`.
- لكن `CompanyDetailsDrawer` غير مفعّل في `CompanyList` لأنه مستورد سابقًا ومعلّق.

الصفحة أو المكوّن	الوظيفة	المدخلات	الحالة المستخدمة	الملفات المرتبطة
LoginPage	تسجيل دخول المدير	phoneNumber/password	phoneNumber, password, error, isLoading	loginPage.tsx , src/lib/apiClient.ts
Dashboard	عرض مؤشرات عامة وخريطة ومخطط	نطاق تاريخ	dateRange, isCalendarOpen	src/pages/Dashboard.tsx
OverviewCards	بطاقات GMV والعمولة والشركات	بيانات trips/companies	statsData, isLoading	features/dashboard/OverviewCards.tsx
RevenueChart	مخطط الإيرادات	[]RevenuePoint	data, isLoading	features/dashboard/RevenueChart.tsx
LiveMap	خريطة مواقع ثابتة	driverLocations ثابتة	لا توجد state	src/features/dashboard/LiveMap.tsx
CompanyList	جدول الشركات	page/search	companies, pageNum, term	features/companies/CompanyList.tsx
CompanyFormModal	إضافة/ تعديل شركة	company prop	formData, isSubmitting	features/companies/CompanyFormModal.tsx
DriverList	سائقون حسب الشركة	selectedCompanyId	drivers, companies, term	src/features/drivers/DriverList.tsx
DriverRegisterPage	تسجيل سائق وصور	form fields/files	form, companies, loading	src/pages/DriverRegisterPage.tsx
CustomerList	جدول العملاء	page/search	customers, term, pageNum	features/customers/CustomerList.tsx
BookingList	عرض حجوزات المطار	search/direction	bookings, searchTerm, direction	src/features/bookings/BookingList.tsx

الصفحة أو المكوّن	الوظيفة	المدخلات	الحالة المستخدمة	الملفات المرتبطة
FinancialSettlement	حساب التسويات	trips	trips, isLoading	settlements/FinancialSettlement.tsx

إدارة الحالة

المشروع يستخدم `useState` و `useEffect` داخل المكونات، و Zustand للتنبيهات. لا يوجد Redux.

النماذج والتحقق

التحقق يتم غالبًا على مستوى Frontend:

- `required` في `inputs`.
- فحص يدوي في `DriverRegisterPage`.
- فحص نوع وحجم الصور في `ImageUploadField`.
- حدود رقمية مثل `min=0` و `max=100` في `CompanyFormModal`.

لكن لا يوجد Schema validation مثل Zod داخل الكود الحالي.

Responsive Design

Tailwind يستخدم classes مثل:

- `grid-cols-1 md:grid-cols-2`
- `flex-col sm:flex-row`
- `overflow-x-auto`

هذا يجعل الجداول والنماذج قابلة للتعامل مع شاشات مختلفة، لكن Sidebar ثابت بعرض `w-64` و `MainLayout` يستخدم `ml-64`، لذلك تجربة الموبايل ليست مكتملة بالكامل رغم وجود زر Menu في `TopBar`.

9. Backend

هذا القسم غير موجود كمشروع Backend داخل المستودع الحالي.
لكن Frontend يتصل ب Backend خارجي. نقطة الاتصال الأساسية:

```
https://aqaariq.com/marketplace/api/v1
```

أو يمكن تغييرها عبر:

```
VITE_API_BASE_URL
```

جدول API المستخدم فعليًا في الكود

Method	Endpoint في الكود	الوظيفة	البيانات المرسلّة	الاستجابة المتوقعة	يحتاج Authentication
POST	/Auth/admin/login	تسجيل دخول المدير	phoneNumber, password	AuthResponse	لا
GET	/Customer?...	جلب العملاء	pageNum, pageSize, term	ApiGetManyResponse	نعم
DELETE	/Customer/{id}	حذف عميل	id	ApiStatusResponse	نعم
GET	/Company?...	جلب الشركات	pageNum, pageSize, term	ApiGetManyResponse	نعم
POST	/Company	إنشاء شركة	name, reputationScore	ApiGetOneResponse	نعم
PUT	/Company/{id}	تعديل شركة	name, status, reputationScore	ApiStatusResponse	نعم
DELETE	/Company/{id}	حذف شركة	id	ApiStatusResponse	نعم
GET	/Driver/ByCompany/{companyId}?...	جلب سائقي شركة	companyId + pagination	ApiGetManyResponse	نعم

Method	Endpoint في الكود	الوظيفة	البيانات المرسله	الاستجابة المتوقعة	يحتاج Authentication
POST	/Driver	إنشاء / تسجيل سائق	FormData أو JSON	ApiResponse	يعتمد على الكود يضيف إن وجد
DELETE	/Driver/{id}	حذف سائق	id	ApiResponse	نعم
POST	/Auth/driver/request-otp	طلب OTP سائق	phoneNumber	غير محدد Type	لا
POST	/Auth/driver/verify-otp	تحقق OTP سائق	phoneNumber, otp	غير محدد Type	لا
POST	/Auth/customer/request-otp	طلب OTP عميل	phoneNumber	غير محدد Type	لا
POST	/Auth/customer/verify-otp	تحقق OTP عميل	phoneNumber, otp	غير محدد Type	لا
GET	/trips	جلب رحلات للتسويات / المؤشرات	لا شيء	[]Trip	نعم
GET	/companies	جلب دليل شركات قديم	لا شيء	[]Company	نعم
GET	/dashboard/chart	جلب بيانات المخطط	لا شيء	[]RevenuePoint	نعم
GET	/settings	جلب الإعدادات	لا شيء	Partial	نعم
PUT	/settings	حفظ الإعدادات	Settings	غير محدد	نعم

ملاحظة: بعض endpoints مثل /settings و /trips و /dashboard/chart غير موجودة في endpoints/API-Admin.md الذي تم فحصه، لذلك وجودها في الخادم غير مؤكد من الملفات المحلية.

Database .10

لا توجد قاعدة بيانات داخل هذا المستودع.

من شكل البيانات والـ API يمكن استنتاج أن الخادم الخارجي يحتوي Entities مثل:

- Customer
- Company
- Driver
- Trip
- Booking

لكن نوع قاعدة البيانات، الجداول، المفاتيح الأساسية، والعلاقات الفعلية غير موجودة في الكود المحلي. لذلك لا يجب الادعاء بأن المشروع يستخدم PostgreSQL أو MongoDB أو أي قاعدة محددة.

مخطط علاقات محتمل من ناحية Frontend فقط:

```
CompanyModel
└─ DriverModel.companyId

Trip
└─ companyId

CustomerModel
└─ used by customer management endpoints
```

هذا مخطط مبني على Types وليس على Schema قاعدة بيانات حقيقية.

هذا القسم غير مستخدم في المشروع الحالي.

لا توجد مكتبات AI أو Machine Learning أو Computer Vision. لا توجد ملفات تدريب Model، ولا Dataset، ولا Inference، ولا Metrics مثل Precision أو Recall أو mAP.

إذا سُئلت أمام اللجنة عن الذكاء الاصطناعي، الإجابة الصحيحة: المشروع الحالي لوحة إدارة وتشغيل، وليس مشروع AI. يمكن مستقبلاً إضافة AI لتوقع الطلب أو كشف الاحتيال أو تحسين توزيع السائقين، لكن هذا غير منفذ حالياً.

12. Hardware and Communication

هذا القسم غير مستخدم في المشروع الحالي.

لا يوجد ESP32 أو GPS مباشر أو كاميرا أو حساسات داخل الكود. الخريطة في `LiveMap.tsx` تعرض نقاط ثابتة وليست بيانات GPS حقيقية.

بروتوكول الاتصال المستخدم فعليًا هو HTTP عبر Fetch API. لا يوجد WebSocket أو Bluetooth أو Wi-Fi device integration داخل المشروع.

13. Installation and Running

المتطلبات

- Node.js مناسب لتشغيل Vite و React.
 - npm.
 - متصفح حديث.
 - اتصال إنترنت للوصول إلى API الخارجي وخريطة OpenStreetMap.
- الإصدارات الدقيقة غير مثبتة في README. من `package.json` المشروع يستخدم:

- React `^19.2.4`
- TypeScript `~5.9.3`
- Vite `^8.1.0`

متغيرات البيئة

اختياري:

```
VITE_API_BASE_URL=https://aqaariq.com/marketplace/api/v1
```

إذا لم يتم ضبطه، يستخدم الكود نفس العنوان كقيمة افتراضية.

خطوات التشغيل

من مجلد المشروع:

```
cd "/Users/isupekira/Desktop/ariport/Dashboard 2/Dashboard"  
npm install  
npm run dev
```

ثم افتح:

```
http://localhost:5175
```

للبناء:

```
npm run build
```

للمعاينة بعد البناء:

```
npm run preview
```

تجربة أهم ميزة

1. افتح `/login`.
2. أدخل بيانات المدير.
3. افتح صفحة Companies.
4. جرب البحث أو إضافة شركة.
5. افتح Drivers واختر شركة.
6. اضغط Register Driver وارفع الصور المطلوبة.

14. Testing

لا توجد اختبارات Unit أو Integration واضحة داخل المشروع. يوجد ملف:

```
test-render.tsx
```

هذا الملف يستخدم `happy-dom` و `renderToString` لمحاولة Render التطبيق، لكنه ليس مدمجًا في `package.json` كسكربت اختبار.

اختبارات مقترحة

النوع	ماذا يختبر
Unit Tests	<code>isSoftDeleted</code> , <code>buildPaginationParams</code> , حسابات التسويات
Component Tests	<code>CompanyList</code> , <code>DriverList</code> , <code>BookingList</code>
API Tests	نجاح وفشل <code>fetchApi</code> , 401 redirect
Form Tests	تسجيل السائق ورفض الصور غير الصحيحة
UI Tests	تسجيل الدخول والتنقل بين الصفحات
Integration Tests	إنشاء شركة ثم ظهورها في الجدول

لا يمكن الادعاء أن المشروع جاهز للإنتاج اعتمادًا على الاختبارات الحالية فقط.

15. Security and Code Quality

نقاط القوة

- استخدام TypeScript Types لبيانات API.
- توحيد الاتصال بالخادم في `fetchApi`.
- عدم وضع قيم سرية واضحة داخل الكود. توجد فقط Base URL عام.
- التعامل مع 401 بإزالة token وإرجاع المستخدم إلى Login.
- استخدام FormData بطريقة صحيحة للصور دون فرض Content-Type يدوي.
- تنظيف Object URLs في `ImageUploadField`.

ملاحظات الجودة والأمان

الأولوية	المشكلة	المسار	سبب أهميتها	الحل المقترح
عالية	حماية الصفحات تعتمد على وجود token فقط	<code>src/App.tsx</code>	token منتهي أو مزور قد يسمح بعرض الواجهة حتى أول طلب API	إضافة تحقق من صلاحية token أو endpoint للتحقق من الجلسة
عالية	بعض رسائل فشل الجلب لا تظهر للمستخدم	<code>CompanyList</code> , <code>CustomerList</code> , <code>DriverList</code>	المستخدم قد يرى Loading ينتهي بدون تفسير	عرض Toast عند فشل جلب البيانات
عالية	صفحة تسجيل السائق لا تنفذ OTP رغم التعليق	<code>src/App.tsx</code> , <code>src/pages/DriverRegisterPage.tsx</code>	قد يتم تقديم تدفق غير موجود	إما تنفيذ OTP أو تعديل النصوص والتعليقات
متوسطة	حجوزات المطار بيانات وهمية	<code>src/services/adminService.ts</code>	قد يظن المستخدم أنها بيانات حقيقية	ربطها بـ API أو وسمها بوضوح كـ Preview

الأولوية	المشكلة	المسار	سبب أهميتها	الحل المقترح
متوسطة	Real- الخريطة ليست time	<code>src/features/dashboard/LiveMap.tsx</code>	النص الظاهر قد يكون مضللاً	ربطها ببيانات مواقع حقيقية أو تغيير التسمية
متوسطة	اختلاف endpoints بين الكود وتوثيق API	<code>adminService.ts</code> , <code>endpoints/API-Admin.md</code>	قد يسبب 404 أو ارتباك أثناء العرض	توحيد أسماء endpoints مع Swagger/Backend
متوسطة	SettingsPage غير مربوطة في App routes الحالية	<code>src/pages/SettingsPage.tsx</code> , <code>src/App.tsx</code>	صفحة موجودة لكن غير قابلة للوصول من التنقل	إضافة Route ورابط أو حذفها من العرض
متوسطة	Settlements موجودة في App لكنها غير موجودة في Sidebar	<code>src/App.tsx</code> , <code>src/layouts/Sidebar.tsx</code>	يصعب الوصول إليها من الواجهة	إضافة رابط أو توضيح أنها غير مفعلة
متوسطة	حقول في CompanyFormModal لا ترسل إلى API	<code>src/features/companies/CompanyFormModal.tsx</code>	واجهة ترحي بحفظ البريد والعنوان والسجل لكن لا تحفظها	ربط الحقول بـ state وAPI أو إزالتها
منخفضة	search TopBar غير فعال	<code>src/layouts/TopBar.tsx</code>	تجربة المستخدم قد تتوقع بحثاً عاماً	ربطه ببحث فعلي أو تعطيله بصرياً
منخفضة	لا توجد اختبارات رسمية	المشروع كامل	صعوبة ضمان عدم كسر الميزات	إضافة Vitest/Testing Library/Playwright

16. Limitations and Future Work

القيود الحالية

- لا يوجد Backend محلي داخل المشروع.
- بعض البيانات Preview أو Mock.
- لا توجد اختبارات مدمجة.
- الحماية Frontend فقط قبل أول API call.
- لا يوجد إدارة أدوار Authorization داخل الواجهة.
- لا توجد Real-time updates.
- Sidebar غير مكتمل لكل الصفحات الموجودة.
- README ما زال README قالب Vite وليس README خاص بالمشروع.

تطوير مستقبلي

- ربط الحجوزات بـ API حقيقي.
- تنفيذ OTP في صفحة تسجيل السائق أو إزالة الإشارة إليه.
- إضافة صفحة Settings إلى التنقل إذا كانت مطلوبة.
- إضافة WebSocket أو polling لمواقع السائقين الحقيقية.
- إضافة اختبارات Unit و E2E.
- تحسين الموبايل Sidebar.
- إضافة Role-based access إذا كان هناك أكثر من دور إداري.
- إضافة Error Boundary.
- تحسين رسائل الخطأ حسب رد الخادم.
- إضافة Audit log للعمليات الحساسة مثل الحذف.

17. Competition Presentation Script

السلام عليكم، مشروعنا اسمه TransPay Dashboard، وهو لوحة تحكم لإدارة منصة نقل مرتبطة برحلات المطار. المشكلة التي ركزنا عليها هي أن إدارة الشركات والسائقين والعملاء والحجوزات تصبح صعبة عندما تكون البيانات موزعة أو تدار يدويًا، خصوصًا عندما يحتاج فريق العمليات إلى معرفة الحالة بسرعة واتخاذ قرار.

الحل الذي قدمناه هو واجهة إدارية موحدة. المدير يبدأ بتسجيل الدخول، وبعدها يستطيع مشاهدة نظرة عامة على أداء النظام، مثل إجمالي الحركة المالية، العمولات، الشركات النشطة، وعدد الرحلات. كذلك يستطيع إدارة الشركات، عرض العملاء، إدارة السائقين حسب الشركة، وتسجيل سائق جديد مع بيانات السيارة وصور التحقق المطلوبة.

تقنيًا، بنينا الواجهة باستخدام React لأنها مناسبة لتقسيم التطبيق إلى مكونات واضحة، واستخدمنا TypeScript لتقليل الأخطاء وتحديد شكل البيانات القادمة من الخادم. استخدمنا Vite لتشغيل المشروع بسرعة أثناء التطوير، وTailwind CSS لبناء واجهة نظيفة وسريعة التعديل. الاتصال بالخادم يتم عبر طبقة موحدة اسمها `fetchApi`، وهي مسؤولة عن إضافة رمز الدخول ومعالجة أخطاء مثل انتهاء الجلسة.

أهم جزء في المشروع من وجهة نظري هو تنظيم تدفق البيانات. مثلاً في صفحة السائقين، نبدأ بجلب الشركات، ثم يختار المدير شركة معينة، وبعدها نجلب السائقين المرتبطين بها فقط. وفي تسجيل السائق نستخدم `FormData` لأن النموذج لا يحتوي نصوصًا فقط، بل صورًا للهوية والمركبة.

واجهتنا بعض التحديات، أهمها توحيد التعامل مع API، وتمييز الأجزاء الجاهزة عن الأجزاء التي ما زالت Preview مثل الحجوزات والخريطة. لذلك حرصنا أن يكون الكود قابلاً للتوسعة: يمكن استبدال البيانات المؤقتة لاحقًا بـ endpoints حقيقية دون إعادة بناء الواجهة بالكامل.

النتيجة الحالية هي لوحة Frontend عملية لإدارة أجزاء أساسية من منصة النقل. مستقبلاً يمكن تطويرها بإضافة بيانات لحظية لمواقع السائقين، ربط كامل للحجوزات، اختبارات تلقائية، وتحسينات أمنية مثل التحقق من صلاحية الجلسة وإضافة صلاحيات متعددة للمديرين.

باختصار، المشروع يحول إدارة منصة نقل من متابعة متفرقة إلى لوحة واحدة منظمة، ويضع أساسًا قابلاً للتطوير نحو نظام تشغيل فعلي ومتكامل.

18. Demo Plan

1. افتح المشروع على `/login`.
2. قل: "هذه بوابة المدير، ولا يمكن الوصول للوحة بدون token".
3. سجل الدخول ببيانات المدير المتاحة.
4. افتح Dashboard ووضح البطاقات والمخطط والخريطة.
5. قل بوضوح: "الخريطة حاليًا تعرض بيانات ثابتة كنموذج عرض، ويمكن ربطها لاحقًا ببيانات GPS".
6. افتح Companies واعرض البحث وAdd Company.
7. افتح Drivers واختر شركة من القائمة.
8. اضغط Register Driver واشرح الحقوق ورفع الصور.
9. افتح Customers واعرض البحث والحذف.
10. افتح Bookings ووضح الفلاتر بين Home to Airport وAirport to Home، واذكر أنها Frontend preview حاليًا.

إذا فشل الإنترنت أو الخادم

- اعرض الكود محليًا واشرح `apiClient.ts`.
- اعرض صفحات Preview مثل Bookings لأنها لا تحتاج API.
- حضر Screenshots مسبقة للصفحات التي تحتاج Backend.
- وضح أن فشل API خارجي لا يعني فشل بنية Frontend.

19. Expected Questions and Answers

أسئلة عامة

1. ما فكرة المشروع؟
لوحة إدارة لمنصة نقل مطار تساعد المدير على متابعة الشركات والسائقين والعملاء والحجوزات والمؤشرات.
2. ما المشكلة التي يحلها؟
يقلل التشتت اليدوي في متابعة العمليات ويجمع الإدارة في لوحة واحدة.
3. ما الذي يميز المشروع؟
تنظيم واضح للعمليات الأساسية مع واجهة قابلة للتوسع وربط API موحد.
4. هل المشروع كامل إنتاجيًا؟
لا أستطيع ادعاء ذلك؛ هناك أجزاء Preview ولا توجد اختبارات كافية بعد.

أسئلة الكود

5. ما نقطة بداية التطبيق؟
src/main.tsx ثم src/App.tsx .
6. أين يتم تعريف المسارات؟
في src/App.tsx باستخدام React Router.
7. كيف تحمون الصفحات الخاصة؟
عبر ProtectedRoute الذي يفحص وجود token في sessionStorage .
8. أين يتم الاتصال بالخادم؟
في src/lib/apiClient.ts عبر fetchApi .
9. لماذا استخدمتم TypeScript؟
لتحديد شكل بيانات API وتقليل الأخطاء أثناء التطوير.

Frontend

10. كيف قسمت الواجهة؟
صفحات في src/pages وميزات في src/features ومكونات مشتركة في src/components/ui .
11. كيف تعرضون Loading؟
عبر state مثل isLoading و Spinner من lucide-react أو Loader مخصص.

12. كيف تعرضون الأخطاء؟

بعض العمليات تعرض Toast، وبعضها يسجل الخطأ فقط في Console وهذه نقطة تحسين.

13. هل الواجهة Responsive؟

جزئيًا باستخدام Tailwind، لكن Sidebar على الموبايل يحتاج تحسينًا.

API و Backend

14. هل يوجد Backend داخل المشروع؟

لا، المشروع Frontend ويتصل بـ API خارجي.

15. كيف تضيفون Authorization؟

يضيف `fetchApi` `Authorization: Bearer <token>` لكل endpoint غير `auth`.

16. ماذا يحدث عند انتهاء الجلسة؟

إذا رجع الخادم 401، يحذف التطبيق token ويحول المستخدم إلى `/login`.

17. لماذا توجد endpoints مختلفة بين الكود والتوثيق؟

هذا اختلاف يجب مراجعته مع Swagger/Backend؛ الكود الحالي يستخدم `/Driver` و `/Company`.

قاعدة البيانات

18. ما قاعدة البيانات المستخدمة؟

غير واضحة من هذا المستودع؛ لا يوجد Database code محلي.

19. ما العلاقات الظاهرة من الواجهة؟

السائق مرتبط بشركة عبر `companyId`، والرحلة مرتبطة بشركة عبر `companyId`.

AI

20. هل يستخدم المشروع AI؟

لا، لا توجد مكتبات أو نماذج AI في الكود الحالي.

21. كيف يمكن إضافة AI مستقبلاً؟

توقع الطلب، تحسين توزيع السائقين، أو كشف العمليات المشبوهة.

الهاردوير والاتصال

22. هل الخريطة تستخدم GPS حقيقي؟

لا، المواقع الحالية ثابتة داخل `LiveMap.tsx`.

23. هل يوجد WebSocket؟

لا، الاتصال الحالي HTTP فقط.

الأمن

24. أين يخزن token؟

في `sessionStorage`.

25. هل هذا كافٍ للإنتاج؟

يحتاج مراجعة أمنية أعمق، مثل التحقق من صلاحية token وإدارة `refresh/session`.

26. هل توجد أسرار داخل الكود؟

لم أجد مفاتيح سرية، لكن يوجد Base URL عام.

الأداء

27. كيف حسنتم حجم التحميل الأول؟

باستخدام Lazy loading للصفحات وبعض المكتبات مثل jsPDF.

28. لماذا قسمت Vite chunks؟

يفصل React و charts و map و pdf لتقليل أثر المكتبات الكبيرة. `vite.config.ts`

التحديات

29. ما أكبر نقطة ضعف؟

وجود أجزاء Preview وعدم وجود اختبارات كافية.

30. ماذا تفعل لو توفر وقت إضافي؟

أربط الحجوزات والخريطة ببيانات حقيقية، أضيف اختبارات، وأحسن الحماية والتنقل.

31. ما الجزء الذي نفذته أنت؟

يمكنك الإجابة حسب دورك الحقيقي. مثال صياغة: "ركزت على واجهة الإدارة، ربطت الخدمات بال API، وتنظيم صفحات الشركات والسائقين والحجوزات."

32. كيف تعرف أن النموذج يعمل بصورة صحيحة؟

حاليًا عبر التجربة اليدوية والبناء، لكن الأفضل إضافة اختبارات تلقائية و E2E.

33. لماذا لم تستخدموا Angular؟

React أخف وأنسب لفريق يريد تقسيم الواجهة بسرعة إلى Components، بينما Angular إطار أكبر.

34. لماذا Zustand بدل Redux؟

لأن الحالة المشتركة في المشروع بسيطة، مثل Toasts، ولا تحتاج بنية Redux كاملة.

35. ما الذي يحدث عند إدخال بيانات خاطئة؟

بعض الحقول لديها `required` وبعضها يتحقق يدويًا، لكن التحقق النهائي يجب أن يكون في Backend أيضًا.

Glossary .20

المصطلح	الترجمة	الشرح المبسط	استخدامه داخل المشروع
Frontend	الواجهة الأمامية	الجزء الذي يراه المستخدم	كل مجلد <code>src</code> تقريبًا
Backend	الخادم	الجزء الذي يعالج البيانات في السيرفر	خارجي فقط، غير موجود محليًا
SPA	تطبيق صفحة واحدة	تطبيق ينتقل داخليًا دون تحميل صفحات HTML جديدة	<code>React Router</code>
Component	مكوّن	جزء مستقل من الواجهة	<code>CompanyList</code> , <code>Button</code>
Props	خصائص	بيانات تمرر من مكون لآخر	<code>CompanyFormModal</code> يستقبل <code>company</code>
State	حالة	بيانات تتغير وتحديث الواجهة	<code>isLoading</code> , <code>companies</code>
Hook	خطاف React	دالة مثل <code>useState</code> و <code>useEffect</code>	معظم المكونات
API	واجهة برمجة التطبيقات	طريقة تواصل Frontend مع الخادم	<code>fetchApi</code>
Authentication	التحقق من الهوية	التأكد من أن المستخدم مسجل	Login + token
Authorization	الصلاحيات	تحديد ما يسمح للمستخدم بفعله	غير مفصل داخل الواجهة
Token	رمز جلسة	نص يستخدم لإثبات الدخول	<code>sessionStorage</code>
CRUD	إنشاء/قراءة/تعديل/حذف	عمليات إدارة البيانات الأساسية	شركات، عملاء، سائقي
Pagination	تقسيم الصفحات	جلب البيانات على دفعات	<code>PaginationControls</code>
FormData	بيانات نموذج متعددة	طريقة إرسال ملفات وصور	تسجيل السائق
Lazy Loading	تحميل مؤجل	تحميل الصفحة عند الحاجة	<code>lazy</code> في <code>App.tsx</code>
Soft Delete	حذف ناعم	اعتبار العنصر محذوفًا بدون إزالته فعليًا	<code>isSoftDeleted</code>
GMV	إجمالي قيمة المعاملات	مجموع قيمة الرحلات	<code>OverviewCards</code>
Commission	عمولة	نسبة المنصة من المبالغ	15% في التسويات
Mock Data	بيانات وهمية	بيانات للعرض بدل API حقيقي	Bookings, LiveMap

المصطلح	الترجمة	الشرح المبسط	استخدامه داخل المشروع
Environment Variable	متغير بيئة	إعداد خارجي للتشغيل	VITE_API_BASE_URL

Final Cheat Sheet .21

فكرة المشروع

TransPay Dashboard هو Frontend لإدارة منصة نقل مطار: شركات، سائقون، عملاء، حجوزات، ومؤشرات مالية.

المشكلة والحل

المشكلة: إدارة العمليات يدويًا ومتفرقة.

الحل: لوحة مركزية فيها جداول، بحث، فلاتر، تسجيل سائق، وحسابات مالية.

أهم التقنيات

- React للواجهة.
- TypeScript للأمان النوعي.
- Vite للتشغيل والبناء.
- Tailwind للتنسيق.
- Zustand للتنبيهات وبعض الحالة.
- Fetch API للاتصال بالخادم.
- Recharts للمخططات.
- React Leaflet للخريطة.
- jsPDF للتصدير.

أهم الملفات

- `src/main.tsx`: تشغيل React.
- `src/App.tsx`: Routes والحماية.
- `src/lib/apiClient.ts`: الاتصال بال API.
- `src/services/adminService.ts`: خدمات الشركات والسائقين والعملاء والحجوزات.
- `src/pages/LoginPage.tsx`: تسجيل الدخول.
- `src/features/companies/CompanyList.tsx`: إدارة الشركات.
- `src/features/drivers/DriverList.tsx`: إدارة السائقين.

- `src/pages/DriverRegisterPage.tsx` : تسجيل سائق.
- `src/features/bookings/BookingList.tsx` : عرض حجوزات Preview.

مسار البيانات العام

Component -> Service Function -> fetchApi -> External API -> JSON -> State -> UI

أهم الدوال

- `fetchApi` : طلبات HTTP ومعالجة token والأخطاء.
- `adminLogin` : دخول المدير.
- `getCompanies` : جلب الشركات.
- `getDriversByCompany` : جلب السائقين حسب الشركة.
- `registerDriver` : إرسال بيانات وصور السائق.
- `isSoftDeleted` : تحديد هل العنصر محذوف ناعماً.
- `buildPaginationParams` : بناء Query String للصفحات.

نقاط القوة

- فصل واضح بين Pages وFeatures وServices.
- TypeScript interfaces لبيانات API.
- Lazy loading للصفحات الثقيلة.
- مكونات UI مشتركة.
- تجربة إدارة واضحة للشركات والسائقين.

القيود

- لا يوجد Backend محلي.
- لا توجد قاعدة بيانات محلية.
- لا يوجد AI أو Hardware.
- لا توجد اختبارات رسمية.
- بعض الميزات Preview أو غير مربوطة.
- الخريطة ليست Real-time.

أكثر 10 أسئلة متوقعة

1. هل يوجد Backend؟ لا، API خارجي.
2. أين يبدأ التطبيق؟ `src/main.tsx`.
3. أين Routing؟ `src/App.tsx`.
4. كيف يتم Login؟ `adminLogin` ثم حفظ token.
5. أين تضيفون Authorization؟ في `fetchApi`.
6. هل الخريطة حقيقية؟ لا، بيانات ثابتة حالياً.
7. هل الحجوزات من API؟ لا، Preview ثابت حالياً.
8. هل يوجد AI؟ لا.
9. ما أكبر نقطة ضعف؟ نقص الاختبارات وبعض التكاملات غير المكتملة.
10. ماذا ستطور لاحقاً؟ ربط Real-time/Bookings، اختبارات، وتحسين الأمان.